

LITE LLM 키 사용 안내 가이드

최종 수정 2026-09-21

LLM 게이트웨이 키 사용 안내

📄 Abstract

발급받으신 키로 LLM 게이트웨이를 사용하는 방법을 안내드립니다. 연결 확인부터 코드 작성, 자주 만나는 오류 해결까지 순서대로 담았습니다. 터미널에서 명령어 한 줄만 실행해 보시면 5분 안에 시작하실 수 있습니다.

📌 <게이트웨이 주소> 는 전달받으신 실제 주소로 바꿔 읽어 주세요

키와 함께 안내드린 `https://...` 형태의 값입니다.

1. 먼저, 받으신 것을 확인해 주세요

시작하는 데 필요한 건 아래 세 가지입니다.

항목	예시	어디에 쓰나요
게이트웨이 주소	<code>https://<게이트웨이 주소></code>	<code>base_url</code>
내 키	<code>sk-</code> 로 시작하는 문자열	<code>api_key</code>
사용 가능한 모델	<code>bedrock-haiku</code> · <code>bedrock-sonnet</code> · <code>bedrock-gpt-5.6-luna</code>	<code>model</code>

⊗ 키는 다시 꺼내 드릴 수 없습니다

보안상 키를 해시로만 저장하기 때문에, 분실하시면 새로 발급받으셔야 합니다. 받으신 즉시 비밀번호 관리자에 저장하시거나 `.env` 파일로 옮겨 두시길 권해 드립니다

다.

2. 5분 안에 연결 확인하기

코드를 작성하기 전에 터미널에서 먼저 확인해 보시면 좋습니다. 키를 매번 붙여넣지 않도록 환경변수에 넣어 두겠습니다.

```
export LLM_BASE_URL="https://<게이트웨이 주소>"
export LLM_API_KEY="sk-발급받은-키"
```

① 사용 가능한 모델 목록 확인

```
curl -sS $LLM_BASE_URL/v1/models \
  -H "Authorization: Bearer $LLM_API_KEY" | jq -r '.data[].id'
```

아래처럼 세 줄이 나오면 정상입니다.

```
bedrock-haiku
bedrock-sonnet
bedrock-gpt-5.6-luna
```

② 실제로 답변이 오는지 확인

```
curl -sS $LLM_BASE_URL/v1/chat/completions \
  -H "Authorization: Bearer $LLM_API_KEY" \
  -H "Content-Type: application/json" \
  -d '{"model": "bedrock-haiku", "messages": [{"role": "user", "content": "안녕이라고만"}]' \
  | jq -r '.choices[0].message.content'
```

답변이 출력되면 준비는 끝입니다. 이제 코드에 연결해 보겠습니다.

① 인증서는 정상 발급된 것이니 안심하셔도 됩니다

주소가 IP 형태여도 공인 인증서(Let's Encrypt)를 사용합니다. curl -k 나 Python 의 verify=False 같은 인증서 검증 우회는 필요하지 않으며, 보안상 사용하지 않으시길 권합니다.

키는 맞는 것 같은데 401 이 날 때

가장 흔한 원인은 눈에 보이지 않는 문자가 키에 섞여 들어간 경우입니다. 화면에는 똑같아 보여도 인증에 실패합니다.

원인	어떻게 섞이나	증상
줄바꿈 문자 Wr	파일을 편집기로 열었다가 복사	글자는 같아 보이는데 401
전각·유사 문자 (: / -)	한글 입력기가 켜진 상태로 입력	Could not resolve host 또는 401

먼저 길이를 확인해 주세요. 키는 **25자**입니다.

```
printf '%s' "$LLM_API_KEY" | wc -c      # 25 가 나와야 정상
```

숫자가 다르면 키에 들어갈 수 있는 문자만 남기고 다시 넣어 주세요.

```
unset LLM_API_KEY
export LLM_API_KEY=$(printf '%s' '여기에-키-붙여넣기' | tr -cd 'A-Za-z0-9_-')
printf '%s' "$LLM_API_KEY" | wc -c      # 다시 25 확인
```

25 가 나오면 해결된 것입니다. 24 이하로 줄어들면 이상 문자가 원래 글자를 대체한 경우라 세탁으로는 고칠 수 없으니, 키를 다시 받아 주세요.

주소도 같은 이유로 깨질 수 있습니다. Could not resolve host 가 나오면 주소를 붙여 넣지 마시고 **직접 입력**해 보세요.

```
unset LLM_BASE_URL
export LLM_BASE_URL="https://<게이트웨이 주소>"      # 손으로 입력
curl -sS "$LLM_BASE_URL/v1/models" -H "Authorization: Bearer $LLM_API_KEY" | jq
```

❗ 변수는 따옴표로 감싸 주세요

`curl -sS $LLM_BASE_URL/v1/models` 처럼 따옴표가 없으면 값에 공백이 섞였을 때 인자가 쪼개집니다. "\$LLM_BASE_URL/v1/models" 로 감싸면 안전합니다.

❗ 키는 문서를 거치지 않고 받으시는 편이 안전합니다

워드·한글·PDF·이미지를 거쳐 옮겨 적으면 위 문자들이 섞이기 쉽습니다. 채팅 코드 블록이나 텍스트 파일로 받아 그대로 복사해 주세요. 입력하실 때 한글 입력기를 꺼두시면 더 안전합니다.

3. 코드에서 사용하기

이 게이트웨이는 **OpenAI 호환 API** 입니다. 기존에 OpenAI 를 쓰시던 코드가 있다면 `base_url` 과 `api_key` 두 줄만 바꾸시면 그대로 동작합니다.

Python

```
pip install openai
```

```
import os
from openai import OpenAI

client = OpenAI(
    base_url=os.environ["LLM_BASE_URL"],    # https://<게이트웨이 주소>
    api_key=os.environ["LLM_API_KEY"],      # sk-...
)

r = client.chat.completions.create(
    model="bedrock-haiku",
    messages=[
        {"role": "system", "content": "너는 간결하게 답하는 도우미다."},
        {"role": "user", "content": "파이썬 리스트 컴프리헨션 한 줄로 설명해줘"}
    ],
```

```
)
print(r.choices[0].message.content)
```

답변이 타이핑되듯 출력되게 하시려면 (스트리밍):

```
stream = client.chat.completions.create(
    model="bedrock-haiku",
    messages=[{"role": "user", "content": "재귀를 3문장으로"}],
    stream=True,
)
for chunk in stream:
    if chunk.choices[0].delta.content:
        print(chunk.choices[0].delta.content, end="", flush=True)
```

Node.js

```
npm install openai
```

```
import OpenAI from "openai";

const client = new OpenAI({
    baseURL: process.env.LLM_BASE_URL,
    apiKey: process.env.LLM_API_KEY,
});

const r = await client.chat.completions.create({
    model: "bedrock-haiku",
    messages: [{ role: "user", content: "안녕" }],
});

console.log(r.choices[0].message.content);
```

그 밖의 도구

base_url (또는 *OpenAI compatible endpoint*)과 API 키를 직접 입력할 수 있는 도구라면 모두 이용하실 수 있습니다.

도구	설정 위치
LangChain	ChatOpenAI(base_url=..., api_key=..., model="bedrock-haiku")
Cursor / Continue / Cline	설정의 <i>OpenAI compatible</i> 항목에 Base URL + Key
Open WebUI, LibreChat	OpenAI API 연결 설정에 동일하게 입력

4. 모델 3종, 어떤 걸 고르면 좋을까요

아래 세 가지 모델을 이용하실 수 있습니다. **모델 이름은 반드시 이 별칭 그대로** 넣어주세요 (claude-3-5-sonnet 같은 원본 ID 로는 호출되지 않습니다).

별칭	실제 모델	성격	이럴 때 추천드립니다
bedrock-haiku	Claude Haiku 4.5	빠르고 가볍습니다	기본값으로 좋습니다. 분류·요약·간단한 변환, 반복 호출
bedrock-sonnet	Claude 3.5 Sonnet v2	균형이 좋습니다	긴 글 작성, 코드 리뷰, 여러 단계로 생각해야 하는 작업
bedrock-gpt-5.6-luna	OpenAI GPT-5.6 Luna	다른 계열입니다	같은 질문을 다른 모델로 교차 확인하고 싶을 때

① **먼저 bedrock-haiku 로 시작해 보세요**

셋 중 가장 가볍고 빠릅니다. 프롬프트를 다듬는 반복 작업은 haiku 로 하시고, 최종 결과만 sonnet 으로 한 번 더 돌리시면 사용량을 아끼면서도 품질을 챙기실 수 있습니다.

5. 사용 한도 안내

발급된 키에는 사용 한도가 설정되어 있습니다.

- 한도에 도달하면 **호출이 자동으로 차단됩니다**. 이때 Budget has been exceeded 메시지가 돌아옵니다.
- 작업이 남아 있는데 차단되셨다면 담당자에게 말씀해 주세요. 확인 후 안내드리겠습니다.

6. 오류가 났을 때

메시지 / 증상	원인	이렇게 해보세요
401 Unauthorized	키가 잘못되었거나, 다른 게이트웨이의 키 입니다	키 문자열에 공백·줄바꿈이 섞이지 않았는지, 주소와 키가 한 쌍인지 확인해 주세요
key not allowed to access model	사용 권한이 없는 모델을 호출하셨습니다	오류 본문에 사용 가능한 목록이 함께 나옵니다. 그중에서 골라 주세요
Budget has been exceeded	사용 한도에 도달했습니다	담당자에게 문의해 주세요
model not found	모델 별칭에 오타가 있습니다	/v1/models 로 정확한 철자를 확인해 주세요
연결이 안 됨 (timeout)	사내망·방화벽 문제이거나 점검 중일 수 있습니다	다른 네트워크에서 curl /v1/models 를 다시 시도해 보신 뒤 담당자에게 알려 주세요
429	짧은 시간에 호출이 몰렸습니다	잠시 후 다시 시도해 주세요. 반복되면 동시 호출 수를 줄여 보세요

⚠ 키는 발급된 게이트웨이에서만 사용할 수 있습니다

다른 분이 쓰시던 예제 코드를 그대로 복사하시면 base_url 이 다른 게이트웨이를 가리켜, 키는 정상인데도 401 이 날 수 있습니다. 주소와 키는 항상 한 쌍으로 관리해 주세요.

7. 안전하게 사용하기 위한 약속

⊗ 아래 네 가지만 지켜 주세요

1. 키를 다른 사람과 공유하지 말아 주세요. 사용량이 키 단위로 집계되어, 함께 쓰면 내 한도가 먼저 소진됩니다.
2. 소스코드나 **Git** 저장소에 키를 넣지 말아 주세요. 환경변수나 `.env` 파일을 사용하고, `.env` 는 `.gitignore` 에 추가해 주세요.
3. 공개 저장소·블로그·스크린샷에 키가 노출되지 않도록 주의해 주세요. 노출된 것 같으면 바로 알려 주시면 즉시 폐기하고 새로 발급해 드립니다.
4. 개인정보나 고객 데이터를 프롬프트에 넣지 말아 주세요. 요청 내용은 사용량 분석을 위해 일정 기간 로그에 보관됩니다.

궁금한 점이나 잘 안 되는 부분이 있으면 SLACK 채널에 편하게 문의해 주세요.

읽기 전용으로 공개된 문서입니다. 링크를 아는 사람만 볼 수 있습니다.